Artificial Intelligence Technology and Application

# Machine Learning Lab Guide

Student Version

HUAWEI

Huawei Technologies CO., LTD.

# Contents

1 Survival Prediction of the Titanic ..... 1
1.1 Introduction ..... 1
1.1.1 About This Lab ..... 1
1.1.2 Objectives ..... 1
1.1.3 Datasets and Frameworks ..... 1
1.2 Procedure ..... 2
1.2.1 Importing Related Libraries ..... 2
1.2.2 Importing Datasets ..... 2
1.2.3 Preprocessing Data ..... 6
1.2.4 Building a Model ..... 8

# 1 Survival Prediction of the Titanic

## 1.1 Introduction

### 1.1.1 About This Lab

This experiment is to predict whether passengers on the Titanic can survive based on the Titanic datasets.

### 1.1.2 Objectives

Upon completion of this task, you will be able to:

- Use the Titanic datasets open to the Internet as the model input data.
- Build, train, and evaluate machine learning models
- Understand the overall process of building a machine learning model.

### 1.1.3 Datasets and Frameworks

This experiment is based on **train.csv** and **test.csv**. **test.csv** contains the result about whether the passengers can survive. This dataset has no target, that is, no result, and can be used as a real-world dataset. Involved parameters are as follows:

- **PassengerId**: passenger ID
- **Pclass**: cabin class (class 1/2/3)
- **Name**: passenger name
- **Sex**: gender
- **Age**: age
- **SibSp**: number of siblings/number of spouses
- **Parch**: number of parents/number of children
- **Ticket**: ticket No.
- **Fare**: ticket price
- **Cabin**: cabin No.
- **Embarked**: port of boarding

# 1.2 Procedure

## 1.2.1 Importing Related Libraries

```
import pandas as pd
import numpy as np
import random as rnd

import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC, LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import Perceptron
from sklearn.linear_model import SGDClassifier
from sklearn.tree import DecisionTreeClassifier
```

## 1.2.2 Importing Datasets

Step 1 Read data.

Step 2 View data.

```
['PassengerId' 'Survived' 'Pclass' 'Name' 'Sex' 'Age' 'SibSp' 'Parch'
 'Ticket' 'Fare' 'Cabin' 'Embarked']
```

The first five rows of data are displayed.

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |

The last five rows of data are displayed.

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.00 | NaN | S |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.00 | B42 | S |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | NaN | 1 | 2 | W./C. 6607 | 23.45 | NaN | S |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.00 | C148 | C |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.75 | NaN | Q |

The data overview helps check whether some data is missing and what the data type is.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Pclass       418 non-null    int64
 2   Name         418 non-null    object
 3   Sex          418 non-null    object
 4   Age          332 non-null    float64
 5   SibSp        418 non-null    int64
 6   Parch        418 non-null    int64
 7   Ticket       418 non-null    object
 8   Fare         417 non-null    float64
 9   Cabin        91 non-null     object
 10  Embarked     418 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 36.0+ KB
```

The related numeric-type information of the data helps check the average value and other statistics.

| | PassengerId | Survived | Pclass | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **count** | 891.000000 | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| **mean** | 446.000000 | 0.383838 | 2.308642 | 29.699118 | 0.523008 | 0.381594 | 32.204208 |
| **std** | 257.353842 | 0.486592 | 0.836071 | 14.526497 | 1.102743 | 0.806057 | 49.693429 |
| **min** | 1.000000 | 0.000000 | 1.000000 | 0.420000 | 0.000000 | 0.000000 | 0.000000 |
| **25%** | 223.500000 | 0.000000 | 2.000000 | 20.125000 | 0.000000 | 0.000000 | 7.910400 |
| **50%** | 446.000000 | 0.000000 | 3.000000 | 28.000000 | 0.000000 | 0.000000 | 14.454200 |
| **75%** | 668.500000 | 1.000000 | 3.000000 | 38.000000 | 1.000000 | 0.000000 | 31.000000 |
| **max** | 891.000000 | 1.000000 | 3.000000 | 80.000000 | 8.000000 | 6.000000 | 512.329200 |

The character-type information helps check the number of types, the type with the maximum value, and the frequency.

| | Name | Sex | Ticket | Cabin | Embarked |
|---|---|---|---|---|---|
| **count** | 891 | 891 | 891 | 204 | 889 |
| **unique** | 891 | 2 | 681 | 147 | 3 |
| **top** | Vovk, Mr. Janko | male | CA. 2343 | B96 B98 | S |
| **freq** | 1 | 577 | 7 | 4 | 644 |

Step 3 Check the survival probability corresponding to each feature based on statistics.

| | Pclass | Survived |
|---|---|---|
| **0** | 1 | 0.629630 |
| **1** | 2 | 0.472826 |
| **2** | 3 | 0.242363 |

The intuitive data shows that passengers in class 1 cabins are more likely to survive.

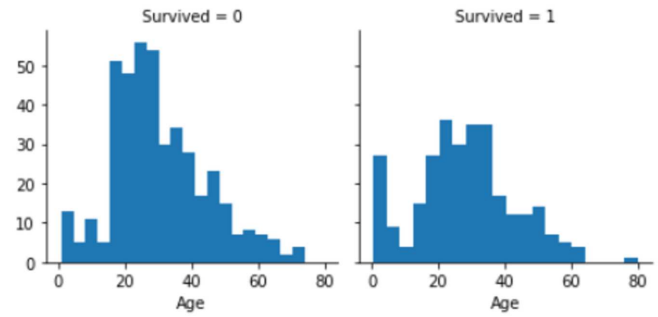
| | SibSp | Survived |
|---|---|---|
| **1** | 1 | 0.535885 |
| **2** | 2 | 0.464286 |
| **0** | 0 | 0.345395 |
| **3** | 3 | 0.250000 |
| **4** | 4 | 0.166667 |
| **5** | 5 | 0.000000 |
| **6** | 8 | 0.000000 |

The survival probability can be directly determined by the number of siblings.

| | Sex | Survived |
|---|---|---|
| **0** | female | 0.742038 |
| **1** | male | 0.188908 |

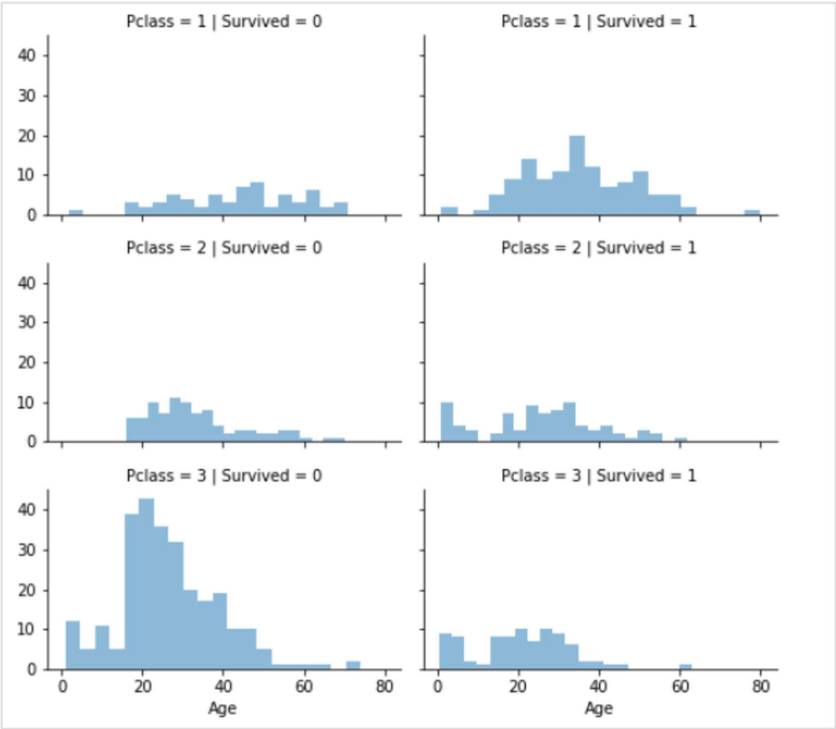
When the survival probability is determined by gender, an obvious imbalance occurs.

```
<seaborn.axisgrid.FacetGrid at 0x1a1df099a88>
```




As shown in the preceding figure, most young passengers died.

The following figure shows the survival probability determined based on the cabin and age.




----End

## 1.2.3 Preprocessing Data

As the datasets have missing values, combine the datasets, and fill the missing values with data.

Step 1 Combine the datasets.

Step 2 Check for missing values.

```
PassengerId       0
Survived        418
Pclass            0
Name              0
Sex               0
Age             263
SibSp             0
Parch             0
Ticket            0
Fare              1
Cabin          1014
Embarked          2
dtype: int64
```

Step 3 Fill the missing values with data.

Process the datasets by using different methods as required. For example, fill the **Fare** and **Embarked** parameters having few missing values with the mode.

Use the average age value.

Delete less significant data. Before this, assign a value to **Target** first.

Check whether missing values still exist.

Step 4 Convert data.

Convert some character-type data into numeric-type data for model input. To do so, check the number of types first.

```
male      843
female    466
Name: Sex, dtype: int64
```

Use the search function to obtain each character-type value and replace it with a numeric-type value.

**test.csv** cannot be used as a training test set as it does not contain **Target**. **train.csv** contains 891 pieces of data (with **Target**), which need to be extracted.

**----End**

## 1.2.4 Building a Model

This section describes how to build a model. To build a model, split the training set and test set.

Step 1 Split the dataset.

```
from sklearn.model_selection import train_test_split
```

Step 2 Train a model.

The logistic regression algorithm, random forest algorithm, and AdaBoost are used for training.

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn import ensemble
```

```
logR 0.7488789237668162
RFC 0.8071748878923767
AdaBoost 0.7757847533632287
```

As shown above, the random forest algorithm has a good effect.

Step 3 Predict data.

```
array([0., 1., 0., 0., 1., 0., 1., 0., 1., 0., 0., 0., 1., 0., 1., 1., 0.,
       0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 0., 0., 0., 0., 0., 1., 1.,
       1., 0., 1., 1., 0., 0., 0., 1., 0., 1., 1., 0., 1., 0., 1., 1., 1.,
       0., 1., 1., 1., 0., 0., 0., 0., 1., 0., 0., 0., 1., 1., 1., 1., 0.,
       1., 1., 1., 0., 1., 1., 1., 1., 0., 1., 0., 1., 1., 0., 0., 0., 0.,
       0., 1., 1., 1., 1., 1., 0., 1., 0., 1., 0., 0., 0., 1., 0., 1., 0.,
       0., 0., 1., 0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 0., 0., 1., 1.,
       1., 1., 0., 1., 0., 0., 1., 0., 1., 0., 0., 0., 1., 0., 0., 0., 0.,
       0., 0., 1., 0., 0., 1., 1., 0., 0., 0., 1., 0., 1., 0., 1., 0., 0.,
       1., 0., 0., 1., 1., 0., 1., 1., 0., 1., 0., 0., 1., 1., 0., 1., 1.,
       0., 0., 0., 0., 0., 1., 1., 1., 1., 0., 0., 1., 1., 0., 1., 0., 1.,
       0., 0., 0., 0., 1., 0., 0., 1., 0., 1., 1., 0., 1., 1., 1., 1., 1.,
       0., 1., 1., 0., 1., 0., 0., 0., 1., 1., 0., 0., 1., 0., 1., 0., 1.,
       0., 1., 0., 1., 1., 0., 1., 0., 0., 1., 1., 0., 0., 1., 0., 0., 0.,
       1., 1., 1., 1., 1., 0., 0., 0., 1., 0., 1., 1., 1., 0., 1., 0., 0.,
       0., 0., 0., 1., 0., 0., 0., 1., 1., 0., 0., 0., 0., 1., 0., 1., 0.,
       1., 1., 0., 1., 0., 0., 0., 0., 1., 1., 1., 1., 1., 0., 0., 1., 0.,
       0., 1., 1., 0., 1., 0., 0., 1., 0., 1., 0., 0., 0., 0., 0., 1., 0.,
       0., 1., 0., 1., 0., 0., 0., 1., 1., 1., 0., 0., 0., 0., 0., 0., 0.,
       1., 1., 0., 1., 1., 0., 0., 1., 1., 0., 1., 0., 1., 0., 0., 0., 0.,
       0., 0., 0., 1., 0., 1., 0., 1., 0., 1., 1., 0., 1., 0., 1., 0., 1.,
       0., 0., 1., 0., 1., 1., 0., 1., 1., 0., 1., 1., 0., 0., 1., 0., 0.,
       1., 1., 1., 0., 0., 0., 0., 0., 1., 1., 0., 1., 0., 0., 0., 0., 1.,
       1., 0., 0., 0., 1., 0., 1., 0., 0., 1., 0., 1., 1., 1., 0., 0., 1.,
       1., 1., 1., 1., 1., 0., 1., 0., 0., 0.])
```

----End